WANDERLUST INTERVIEW GUIDE

Ye PDF WanderLust project ke liye Hinglish interview preparation notes hai. Isme project explanation, architecture, features, auth flow, important code snippets aur likely interview questions cover kiye gaye hain.

1. One-line Introduction

WanderLust ek full-stack Airbnb-style travel listing platform hai jo Node.js, Express, MongoDB aur EJS par build kiya gaya hai.

2. Short Interview Intro

WanderLust mera server-rendered full-stack project hai jisme users signup/login kar sakte hain, listings create/edit/delete kar sakte hain, Cloudinary ke through images upload kar sakte hain, aur Mapbox ke through location geometry generate hoti hai. Is project se maine authentication, authorization, CRUD, validation aur media handling practically seekhi.

3. Tech Stack

- Node.js
- Express 5
- MongoDB + Mongoose
- EJS + ejs-mate
- Passport.js
- express-session
- connect-mongo
- Cloudinary
- Multer
- Mapbox
- Joi

4. Architecture

Project MVC-style structure follow karta hai:
- routes request receive karti hain
- controllers business logic chalate hain
- models data structure define karte hain
- EJS views server-side HTML render karti hain

5. Main Features

- Signup, login, logout
- Session-based authentication
- Listing CRUD
- Listing owner authorization
- Image upload to Cloudinary
- Country search
- Category filtering
- Reviews relation
- Flash messages
- Geocoded location coordinates

6. Important Code with Line Meaning

Snippet A: app startup

```
if (process.env.NODE_ENV !== "production") { // Sirf development me local env load ho
  require("dotenv").config(); // .env file se secrets load ho rahe hain
}

const express = require("express"); // Express framework import hua
const app = express(); // App instance create hui
const mongoose = require("mongoose"); // MongoDB ODM import hua
const session = require("express-session"); // Login session maintain karne ke liye
const passport = require("passport"); // Auth library import hui
const LocalStrategy = require("passport-local"); // Username/password auth strategy
```

Snippet B: session config

```
const store = MongoStore.create({ // Session data MongoDB me save hoga
  mongoUrl: dbUrl, // Database URL use ho rahi hai
  touchAfter: 24 * 3600, // Session DB writes optimize ho rahe hain
});

const sessionConfig = {
  store, // Memory ki jagah Mongo store
  secret: process.env.SESSION_SECRET || "mysupersecretcode", // Session signing secret
  resave: false, // Har request par force save avoid hota hai
  saveUninitialized: true, // New session save behavior
  cookie: {
    httpOnly: true, // Cookie JS se direct access na ho
    maxAge: 7 * 24 * 60 * 60 * 1000, // 1 week validity
  },
};
```

Snippet C: protected listing creation route

```
router.route("/")
  .get(wrapAsync(listingController.index)) // Sare listings fetch/render karne ke liye
  .post(
    isLoggedIn, // Sirf logged-in user create kar sake
    upload.single("listing[image]"), // Form se ek image upload hogi
    validateListing, // Joi validation chalegi
    wrapAsync(listingController.createListing) // Actual DB save logic chalega
  );
```

Snippet D: create listing controller

```
let response = await geocodingClient.forwardGeocode({ // Location ko coordinates me
convert kiya
  query: req.body.listing.location, // User-entered location
  limit: 1 // Best match only
}).send();

let url = req.file.path; // Uploaded image URL
```

```
let filename = req.file.filename; // Uploaded image filename

const newListing = new Listing(req.body.listing); // Listing object create hua
newListing.owner = req.user._id; // Current user owner bana
newListing.image = { url, filename }; // Image metadata assign hui
newListing.geometry = response.body.features[0].geometry; // Coordinates DB me save hui

await newListing.save(); // Listing save ho gayi
```

Snippet E: login check middleware

```
module.exports.isLoggedIn = (req, res, next) => {
  if (!req.isAuthenticated()) { // Passport session se auth check
    req.session.redirectUrl = req.originalUrl; // Login ke baad wapas same page
    req.flash("error", "You must be logged in to do that!"); // User feedback
    return res.redirect("/login"); // Login page redirect
  }
  next(); // Authenticated user next step par jayega
};
```

7. Interview Questions

Q. Passport kyun use kiya?
Answer:
Server-rendered app ke liye session-based auth natural fit tha aur Passport ne login flow ko simplify kiya.

Q. Cloudinary kyun use kiya?
Answer:
Image storage ko local server se alag rakhne ke liye. Isse media management aur deployment easier hota hai.

Q. Mapbox kyun add kiya?
Answer:
Listing location ko coordinates me convert karke future map support aur structured geodata maintain karne ke liye.

8. HR Style Answer

Ye project maine backend depth aur real-world full-stack flow dikhane ke liye banaya. Isme sirf CRUD nahi, balki auth, sessions, ownership, uploads aur validation sab saath me aate hain.

9. Improvements

- Better security hardening
- Pagination
- Test coverage
- Better error pages
- Advanced search and filters